

JAVA

Nested Classes em Java

Static Nested • Inner Classes • Local Classes • Anonymous Classes

Apostila completa dos quatro tipos de classes aninhadas, com explicações detalhadas e exemplos práticos.

CONTEÚDO DA APOSTILA

- 01** Introdução: Os Quatro Tipos
- 02** Static Nested Class
- 03** Inner Classes
- 04** Local Classes
- 05** Anonymous Classes
- 06** Anonymous Class vs. Lambda

Sumário

01 Introdução: Os Quatro Tipos

Quando faz sentido declarar um tipo dentro de outro

02 Static Nested Class

Uma classe independente que só mora dentro de outra

03 Inner Classes

Classes de instância fortemente acopladas à outer

04 Local Classes

Classes declaradas dentro de um bloco de método

05 Anonymous Classes

Classes sem nome, criadas e usadas em uma única expressão

06 Anonymous Class vs. Lambda

Critérios de escolha e erros comuns

01 Introdução: Os Quatro Tipos de Nested Classes

Quando faz sentido declarar um tipo dentro de outro

Em Java, é possível declarar uma classe, interface, enum ou record dentro do corpo de outra classe. Esses tipos aninhados — nested types — fazem sentido quando duas peças de código estão tão entrelaçadas que é mais natural mantê-las juntas do que espalhadas em arquivos separados. Java reconhece quatro variações desse recurso, cada uma com regras próprias de acesso, escopo e uso ideal.

```
public class Estacionamento {

    // ① Static Nested Class – acessada via Estacionamento.NomeDaClasse
    // não carrega referência implícita à instância externa
    static class Recibo { /* ... */ }

    // ② Inner Class – acessada a partir de uma instância
    // carrega referência implícita à instância externa
    class Vaga { /* ... */ }

    public void algumMetodo() {
        // ③ Local Class – existe só dentro deste método
        class Calculo { /* ... */ }

        // ④ Anonymous Class – sem nome, criada e usada na hora
        Runnable tarefa = new Runnable() {
            @Override public void run() { /* ... */ }
        };
    }
}
```

Exemplo — os quatro tipos de nested class dentro de uma mesma classe

Tipo	Onde é declarada	Acesso à outer	Palavra-chave
Static Nested	Corpo da classe	Via instância explícita	static class
Inner Class	Corpo da classe	Direta (implícita)	class (sem static)
Local Class	Dentro de um método	Sim + variáveis capturadas	(sem modificador)
Anonymous Class	Expressão inline	Sim + variáveis capturadas	new Tipo() { }

A PARTIR DO JDK 16

Desde o JDK 16, todos os quatro tipos de nested class podem ter membros static — incluindo métodos estáticos, constantes e outras classes aninhadas estáticas, algo que antes só a Static Nested Class permitia livremente.

02 Static Nested Class

Uma classe independente que só mora dentro de outra

Conceito e forma de acesso

Uma Static Nested Class é declarada com o modificador `static` dentro do corpo de outra classe. Ao contrário de uma Inner Class, ela não carrega nenhuma referência implícita à instância da classe externa — na prática, comporta-se como uma classe independente que apenas vive dentro do espaço de nomes da outer.

Para acessá-la de fora, usa-se `Externa.Aninhada obj = new Externa.Aninhada()`. De dentro da própria classe externa, o nome simples já basta: `new Aninhada()`. Ela é ideal para estruturas auxiliares que não dependem do estado da outer — o exemplo mais conhecido da própria JDK é a classe `Node`, usada internamente por `LinkedList`.

```
public class Estacionamento<T> {  
  
    // Static Nested – Vaga não precisa de uma instância de Estacionamento:  
    private static class Vaga<T> {  
        T ocupante;  
        Vaga<T> proxima;  
        Vaga(T ocupante) { this.ocupante = ocupante; }  
    }  
  
    private Vaga<T> primeira;  
    private int totalOcupadas;  
  
    public void estacionar(T veiculo) {  
        Vaga<T> nova = new Vaga<>(veiculo); // acesso pelo nome simples  
        if (primeira == null) { primeira = nova; }  
        else {  
            Vaga<T> atual = primeira;  
            while (atual.proxima != null) atual = atual.proxima;  
            atual.proxima = nova;  
        }  
        totalOcupadas++;  
    }  
}
```

Exemplo — Static Nested Class formando uma lista encadeada de vagas

Acesso mútuo a membros privados

Um detalhe pouco óbvio, mas muito útil: a Static Nested Class pode acessar membros privados da classe externa, e a classe externa também pode acessar membros privados da nested — o acesso é bidirecional. Isso é possível porque ambas são compiladas como parte do mesmo arquivo `.class` gerado. Esse comportamento é justamente o que sustenta o padrão Builder: a classe interna Builder acessa livremente os campos privados do objeto que está montando.

```
public class Pedido {
    private final String cliente;
    private final double total;

    private Pedido(Builder b) {
        this.cliente = b.cliente; // acessa privado do Builder
        this.total = b.total;
    }

    public static class Builder {
        private String cliente;
        private double total;
        public Builder cliente(String c) { this.cliente = c; return this; }
        public Builder total(double t) { this.total = t; return this; }
        public Pedido build() { return new Pedido(this); }
    }
}

// uso – encadeamento típico do padrão Builder:
Pedido p = new Pedido.Builder().cliente("Marina").total(159.9).build();
```

Exemplo — Static Nested Class implementando o padrão Builder

PREFIRA STATIC NESTED SEMPRE QUE POSSÍVEL

Se a classe aninhada não precisa acessar o estado da instância externa, prefira Static Nested. Ela evita manter uma referência implícita à outer, reduzindo o risco de vazamento de memória em coleções de longa duração.

03 Inner Classes

Classes de instância fortemente acopladas à classe externa

Conceito e referência implícita

Uma Inner Class é uma classe não-estática declarada no nível de membro da outer. A diferença central em relação à Static Nested é que toda Inner Class carrega uma referência implícita à instância da outer que a criou — por isso, ela pode acessar diretamente todos os campos e métodos da outer, inclusive os privados, sem precisar de nenhuma referência explícita.

Dentro da Inner Class, essa referência à outer pode ser acessada explicitamente como `Externa.this` — necessário apenas quando há conflito de nomes entre um campo local e um campo da outer. Como cada instância de uma Inner Class está amarrada a uma instância específica da outer, ela não podia ter membros static antes do JDK 16.

```
public class Estacionamento {
    private String nomeUnidade;
    private List<String> placasEstacionadas = new ArrayList<>();

    // Inner Class — acessa 'placasEstacionadas' diretamente:
    private class RelatorioDeOcupacao implements Iterator<String> {
        private int indice = 0;
        @Override public boolean hasNext() {
            return indice < placasEstacionadas.size(); // campo da outer!
        }
        @Override public String next() {
            return placasEstacionadas.get(indice++);
        }
    }

    public Iterator<String> iterador() {
        return new RelatorioDeOcupacao(); // instanciada de dentro da outer
    }
}
```

Exemplo — Inner Class implementando um Iterator sobre o estado da outer

Instanciando uma Inner Class de fora

De dentro da própria outer, a Inner Class é instanciada normalmente com `new Inner()`. De fora, é preciso primeiro ter uma instância da outer e então usar a sintaxe especial `instanciaExterna.new Inner()`. Essa sintaxe existe porque a Inner Class depende da referência à outer para existir. Na prática, Inner Classes costumam ser `private`, o que torna essa instanciização externa rara — quando ela parece necessária, geralmente é sinal de que uma Static Nested Class resolveria melhor.

```
// de dentro da outer — instanciização simples:
public class Externa {
    class Interna {
        void ola() { System.out.println("Interna de " + Externa.this); }
    }
    void criar() { new Interna().ola(); }
}

// de fora — sintaxe especial com .new:
Externa externa = new Externa();
Externa.Interna interna = externa.new Interna();
interna.ola();
```

Exemplo — instanciando uma Inner Class de dentro e de fora da outer

04 Local Classes

Classes declaradas dentro de um bloco de método

Conceito e escopo limitado

Local Classes são Inner Classes declaradas diretamente dentro de um bloco de código — geralmente um método, mas também podem aparecer em blocos estáticos ou inicializadores de instância. O escopo delas é limitado àquele bloco: nenhum código fora dali sabe que a classe existe ou consegue instanciá-la.

Além de acessar os campos e métodos da outer como qualquer Inner Class, uma Local Class também pode capturar variáveis locais do método onde foi declarada — mas somente se essas variáveis forem `final` ou `effectively final`, ou seja, nunca reatribuídas depois de inicializadas.

```
public class CalculadoraDeTarifa {
    private double valorPorHora = 8.0;

    public void gerarCobranca(int minutosEstacionado) {
        // Local Class — só existe dentro deste método
        class Cobranca {
            // acessa 'valorPorHora' da outer e 'minutosEstacionado' do método:
            double calcular() {
                double horas = minutosEstacionado / 60.0;
                return horas * valorPorHora;
            }
        }
        Cobranca c = new Cobranca();
        System.out.printf("Total a pagar: R$%.2f%n", c.calcular());
    }
}
```

Exemplo — Local Class capturando um parâmetro effectively final do método

A exigência de final ou effectively final

Quando uma Local Class usa uma variável local do método que a contém, o Java copia o valor dessa variável para dentro da instância da Local Class. Isso acontece porque a instância pode continuar viva no heap depois que o método já terminou e sua variável local, que vivia na stack, deixou de existir. Para garantir que essa cópia sempre reflita um valor consistente, o compilador exige que a variável nunca seja reatribuída depois de inicializada.

JDK 16+ EM LOCAL CLASSES

A partir do JDK 16, Local Classes também podem conter records, interfaces e enums locais — todos implicitamente `static` dentro da Local Class que os declara.

05 Anonymous Classes

Classes sem nome, declaradas e instanciadas em uma única expressão

Conceito

Uma Anonymous Class é, na prática, uma Local Class sem nome. Ela é sempre declarada e instanciada ao mesmo tempo, em uma única expressão, com `new` seguido de uma interface a

implementar ou uma classe (abstrata ou concreta) a estender. Como é uma expressão, e não uma declaração, ela precisa terminar com ponto e vírgula.

Antes do Java 8, Anonymous Classes eram o jeito padrão de implementar interfaces funcionais, listeners e comparadores inline. Hoje, lambdas cobrem a maior parte desses casos com muito menos código — mas Anonymous Classes continuam necessárias quando é preciso implementar mais de um método ou manter estado interno próprio.

```
// implementando uma interface com Anonymous Class:
Comparator<String> porTamanhoDaPlaca = new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
}; // ponto e vírgula obrigatório — é uma expressão!

// estendendo uma classe abstrata anonimamente:
abstract class Tarifa {
    abstract double calcular(int minutos);
}

Tarifa tarifaNoturna = new Tarifa() {
    @Override public double calcular(int minutos) {
        return minutos * 0.12; // valor diferenciado à noite
    }
};
```

Exemplo — Anonymous Class implementando uma interface e estendendo uma classe abstrata

Anonymous Class com estado próprio

```
Runnable contadorDeEntradas = new Runnable() {
    private int total = 0; // campo próprio — lambdas não têm campos!
    @Override public void run() {
        total++;
        System.out.println("Entrada registrada: " + total);
    }
};
```

Exemplo — estado interno só é possível em uma Anonymous Class, não em uma lambda

06 Anonymous Class vs. Lambda

Quando cada uma é a escolha certa, e erros comuns para evitar

Desde o Java 8, lambdas substituem a Anonymous Class na maioria dos casos envolvendo interfaces funcionais — interfaces com um único método abstrato. O lambda é mais conciso, mais legível e não gera uma classe extra no bytecode do mesmo jeito que uma Anonymous Class gera. Ainda assim, existem cenários em que a Anonymous Class continua sendo a única opção viável.


```
// lambda – ideal para interfaces funcionais (1 método abstrato):
Runnable r1 = () -> System.out.println("Lambda – direto ao ponto");
Comparator<String> c1 = (a, b) -> a.compareTo(b);

// Anonymous – necessária quando há ESTADO próprio:
Runnable r2 = new Runnable() {
    private int execucoes = 0;
    @Override public void run() { System.out.println(++execucoes); }
};

// Anonymous – necessária para MÚLTIPLOS métodos abstratos:
Iterator<String> it = new Iterator<String>() {
    private String[] placas = {"ABC1234", "XYZ9988"};
    private int idx = 0;
    @Override public boolean hasNext() { return idx < placas.length; }
    @Override public String next() { return placas[idx++]; }
};

// Anonymous – necessária para ESTENDER uma classe abstrata/concreta:
abstract class Notificador { abstract void avisar(String msg); }
Notificador n = new Notificador() {
    @Override public void avisar(String msg) { System.out.println("Aviso: " + msg); }
};
```

Exemplo — os quatro cenários que definem a escolha entre lambda e Anonymous Class

Tipo	Referência à outer	Pode ter estado	Melhor uso
Static Nested	Nenhuma	Sim, próprio	Estruturas auxiliares independentes (Node, Builder)
Inner Class	Implícita e total	Sim, próprio	Iteradores e componentes acoplados à instância
Local Class	Implícita + variáveis capturadas	Sim, próprio	Lógica usada uma única vez dentro de um método
Anonymous Class	Implícita + variáveis capturadas	Sim, próprio	Implementação inline com estado ou múltiplos métodos

ERRO COMUM

Tentar reatribuir, dentro de uma Local ou Anonymous Class, uma variável do método externo que ela já capturou. Isso quebra a exigência de effectively final e o código simplesmente não compila — a solução é usar uma variável auxiliar que não precise mudar depois da captura.

Resumo Final

Static Nested Sem referência à outer. Ideal para estruturas auxiliares independentes.

Inner Class	Referência implícita e total ao estado da outer. Acoplamento forte.
Local Class	Vive dentro de um método; pode capturar variáveis final/effectively final.
Anonymous Class	Sem nome, inline, útil para estado próprio ou múltiplos métodos.
Lambda vs. Anonymous	Lambda para interfaces funcionais simples; Anonymous para estado e múltiplos métodos.